LAB SHEET 9

One-Dimensional Arrays

1. Objectives

- Students are able to understand the concept of one-dimensional arrays and how to declare, initialize, and access their elements in the Java programming language.
- Students are able to write programs that use one-dimensional arrays to store and process data, including the use of for and foreach loops.
- Students are able to apply arithmetic operations to array elements, such as calculating the total, average, highest value, and lowest value.
- Students are able to apply simple searching and sorting algorithms to one-dimensional arrays.
- Students are able to apply one-dimensional arrays to solve case studies that are relevant to campus life and everyday situations.

2. Theoretical Background

2.1. What Is an Array?

An array is a data structure that stores a collection of data of the same type, in sequence, within a single variable. Instead of creating a separate variable for each piece of data (for example, score1, score2, score3, ... score10), an array allows all of the data to be stored in one variable and accessed by index. In Java, arrays are homogeneous (all elements must be of the same type) and fixed in size — the size cannot be increased or decreased once the array has been instantiated.

Internally, array elements are stored contiguously in memory, so the computer can access any element directly by its index without having to traverse the preceding elements one by one. This is one reason why accessing an array element (for example, scores[5]) is very fast compared with other data structures that are not stored contiguously.

2.2. Declaring and Initializing Arrays

There are several ways to declare and initialize a one-dimensional array in Java, depending on whether the element values are known from the start or will be filled in later.

Syntax	Explanation
`int[] numbers = new int[4];`	Declares an empty array with a capacity of 4 elements. All elements are automatically filled with the default value (0 for the int type) until they are explicitly assigned.
`int[] numbers = {5, 13, -7, 17};`	Declares an array and initializes its element values directly. The array size automatically matches the number of values written.
`int[] numbers = new int[]{5, 13, -7, 17};`	Another form of direct initialization, commonly used when an array is declared as a method argument or a return value.
`String[] menu = {"Fried Rice", "Pulled Tea"};`	Arrays can also store non-primitive types such as String, as long as all of the elements are of the same type.

2.3. Indexes, Array Length, and Common Errors

Array indexes in Java always start at 0, so the first element has index 0 and the last element has index (array length − 1). The length or capacity of an array can be obtained through the .length attribute, for example numbers.length. Note that .length is an attribute (not a method), so it is written without parentheses — unlike String, which uses the .length() method.

The most common mistake among beginners is trying to access an index outside the range of the array, for example accessing numbers[4] in an array with a capacity of 4 elements (the valid indexes are only 0 to 3). This mistake causes the program to terminate with an ArrayIndexOutOfBoundsException while it is running (a runtime error), rather than when the program is compiled.

Default Value	Applies to Type
`0 (zero)`	The default value of array elements of type int or double before they are explicitly assigned.
`false`	The default value of array elements of type boolean.
`null`	The default value of array elements of object/reference types, for example String.

2.4. Processing Arrays with Loops

There are several ways to iterate over all of the elements of an array in Java, each with its own use.

Technique	Use
`for (int i = 0; i < arr.length; i++)`	The most flexible approach — the index i can be used to access or modify element values, or to compare an element with another element (for example, arr[i] and arr[i+1]).
`for (int value : arr)`	Foreach loop (enhanced for loop) — more concise for simply reading or displaying each element, but it cannot provide the index of an element directly and cannot be used to modify the contents of the array.
`Arrays.toString(arr)`	A helper method from the java.util.Arrays class that displays the entire contents of an array on a single line in the format [element1, element2, ...], without the need to write a manual loop.
`Arrays.sort(arr)`	A helper method for sorting an array in ascending order using an internally optimized sorting algorithm.

2.5. Common Array Operations: Aggregation, Searching, and Sorting

The three most common operations applied to arrays are:

- Aggregation (total, average, maximum/minimum value) — performed by traversing all of the elements while accumulating the result in a single variable (for example, total) or comparing each element with a temporary holder variable (for example, highest).

- Searching — finding out whether a value exists in an array and, if so, at which index it is located. The simplest technique is linear search, which examines the elements one by one from the beginning until the value is found or all of the elements have been checked.
- Sorting — rearranging the elements of an array in a particular order (ascending or descending). One of the simplest sorting algorithms to learn is bubble sort, which works by repeatedly comparing two adjacent elements and swapping their positions.

To give an idea of efficiency (not to be memorized, only to build intuition), the following are the worst-case time complexities of each technique, expressed in Big-O notation — the larger n (the number of array elements) becomes, the more noticeable the difference in efficiency is:

Operation	Time Complexity (Big-O)
`Accessing an element by index (arr[i])`	O(1) — constant time, regardless of the size of the array.
`Linear search`	O(n) — time grows in direct proportion to the number of elements, because the elements are checked one by one.
`Bubble sort`	$O(n^2)$ — time grows quadratically with the number of elements, because there are two nested loops.

2.6. Array vs. ArrayList: When to Use Which?

Java also provides the ArrayList data structure (part of the Java Collections Framework), which is similar to an array but can grow or shrink dynamically according to the needs of the program while it is running.

Aspect	Comparison
`Size`	An array is fixed in size from the moment it is instantiated. An ArrayList can grow or shrink dynamically.
`Element type`	An array can hold primitive types (int, double, etc.) directly. An ArrayList can only hold object types (Integer, Double, etc. must be used).
`Performance`	Arrays are generally slightly faster and more memory-efficient because their structure is simpler.
`Ease of use`	ArrayList provides many ready-to-use methods (add, remove, contains, etc.) that make data processing easier than with arrays.

As a general rule, use an array when the amount of data is known and will not change while the program is running (for example, the number of days in a week, or a fixed number of menu items). Use an ArrayList when the amount of data may grow or shrink dynamically (for example, a list of orders that keeps growing). ArrayList will be discussed in more detail in the next lab sheet.

2.7. Relevance of One-Dimensional Arrays to the Lab Case Studies

In this lab, one-dimensional arrays are applied to a case study relevant to the campus environment, namely the management of student scores in the Academic Information System (SIAKAD) — including score entry, determination of pass/fail status, and the calculation of statistics such as the average and the class grade category. One-dimensional arrays are also applied to a case study on order management at a campus café, covering the recording of the menu list, menu searching, order recording, and total cost calculation.

Both case studies represent usage patterns of arrays that are very common in real-world applications: storing a collection of similar data (student scores, a menu list, a price list), and then processing it using a combination of loops, aggregation, searching, and sorting, as explained in the previous sections.

Running the Program in VS Code: All program code in this lab sheet is written and run using Visual Studio Code. Create a new file with the name requested in each step (File > New File, then save it using Save As with the appropriate name and the .java extension). To compile and run the program, open the integrated VS Code terminal via the Terminal > New Terminal menu or the shortcut Ctrl + ` (backtick), then run the command javac FileName.java to compile, followed by java FileName (without .java) to run the program.

3. Lab Exercises

3.1. Experiment 1: Declaring, Initializing, Filling, and Displaying Array Elements

Experiment Duration: 30 minutes

1. Open VS Code. Create a new file (**File > New File**), then save it with the name **NumberArray<RollNumber>.java**
2. Create an integer array named numbers with a capacity of 4 elements:

```
int[] numbers = new int[4];
```

3. Fill each element of the numbers array with the values 5, 13, -7, 17:

```
numbers[0] = 5;
numbers[1] = 13;
numbers[2] = -7;
numbers[3] = 17;
```

4. Display all of its elements on the screen using a for loop:

```
for (int i = 0; i < numbers.length; i++) {
    System.out.println("numbers[" + i + "] = " + numbers[i]);
}
```

5. As an enrichment exercise, also display the contents of the array using a foreach loop, and compare it with displaying the array directly using the helper method Arrays.toString():

```
for (int number : numbers) {
    System.out.println(number);
}

import java.util.Arrays;
System.out.println(Arrays.toString(numbers));
```

6. Open the VS Code terminal (Ctrl + `), then compile and run the program with the following commands: javac NumberArray<RollNumber>.java and then java NumberArray<RollNumber>. Observe the results and check that they match the reference.

Experiment 1 Sample Output (Reference)



Example of the VS Code terminal display after the program is run. The results on each student's laptop must match the display above.

Experiment 1 Questions

1. If the elements of the numbers array are changed to 5.0, 12867, 7.5, 2000000, what happens? Why does this happen?
2. Modify the program above by initializing the array elements at the same time the array is declared (without new int[4] and without filling the elements one by one).
3. Change the display statement in step 4 to the following: System.out.println("numbers[" + i + "] = " + numbers[i] + ", doubled = " + (numbers[i] * 2)); What is the output of the program? Explain the meaning of the statement.
4. If the condition in the for loop above is changed to i <= 4, what is the output of the program? Why?
5. What is the difference between displaying an array using a for loop, a foreach loop, and Arrays.toString() in step 5? When should each one be used?

3.2. Experiment 2: SIAKAD Case Study — Entering Student Scores and Pass/Fail Status

Experiment Duration: 45 minutes

In the Academic Information System (SIAKAD), lecturers need to enter students' final scores and quickly determine the pass/fail status of each student.

1. Open VS Code. Create a new file and save it with the name **ScoreArray<RollNumber>.java**
2. Create the basic Java structure (the class and the main method), then import the Scanner library.
3. Declare an integer array named finalScores with a capacity of 10 elements:

```
int[] finalScores = new int[10];
Scanner input = new Scanner(System.in);
```

4. Create a loop structure to receive input and fill the elements of the finalScores array:

```
for (int i = 0; i < finalScores.length; i++) {
    System.out.print("Enter the score for student #" + (i + 1) + ": ");
    finalScores[i] = input.nextInt();
}
```

5. Using a loop structure, display all of the elements of the finalScores array along with each student's pass/fail status (PASS if the score is greater than 70, FAIL otherwise):

```
for (int i = 0; i < finalScores.length; i++) {
    String status = (finalScores[i] > 70) ? "PASS" : "FAIL";
    System.out.println("Student #" + (i + 1) + ": " + finalScores[i] + " (" +
status + ")");
}
```

6. Open the VS Code terminal, then compile and run the program with the following commands: javac **ScoreArray<RollNumber>.java** and then java **ScoreArray<RollNumber>**. Observe the results by entering several different scores.

Experiment 2 Sample Output (Reference)

The following is a sample output when the program is run with these input scores: 85, 60, 72, 90, 45, 78, 66, 95, 50, 71 (students may use other scores, but the output must follow the same format).

```
TERMINAL – ScoreArray01

$ javac ScoreArray01.java
$ java ScoreArray01
Enter the score for student #1: 85
Enter the score for student #2: 60
Enter the score for student #3: 72
Enter the score for student #4: 90
Enter the score for student #5: 45
Enter the score for student #6: 78
Enter the score for student #7: 66
Enter the score for student #8: 95
Enter the score for student #9: 50
Enter the score for student #10: 71
Student #1: 85 (PASS)
Student #2: 60 (FAIL)
Student #3: 72 (PASS)
Student #4: 90 (PASS)
Student #5: 45 (FAIL)
Student #6: 78 (PASS)
Student #7: 66 (FAIL)
Student #8: 95 (PASS)
Student #9: 50 (FAIL)
Student #10: 71 (PASS)
```

Example of the VS Code terminal display after the program is run with the sample data above.

Experiment 2 Questions

1. What does the condition i < finalScores.length in the loop above mean?
2. Modify the program so that it displays only the scores of students who passed (score > 70), without displaying those who failed. Explain the flow of the program!
3. Change the loops in steps 4 and 5 from for loops to foreach loops. Can all parts of the program be converted to foreach? Explain which parts can and cannot be converted, along with the reasons.
4. Modify the program so that it also calculates and displays the number of students who passed and the number of students who failed at the end of the program.

3.3. Experiment 3: Arithmetic Operations on Arrays — Total, Average, Highest and Lowest Scores

Experiment Duration: 45 minutes

This experiment continues the SIAKAD case study to produce a summary of student scores, as commonly shown in an end-of-semester score recap.

1. Open VS Code. Create a new file and save it with the name **AverageScoreArray<RollNumber>.java**
2. Create the basic Java structure, then import and declare Scanner for user input.
3. Create an integer array named studentScores with a capacity of 10, and declare the variables total and average:

```
int[] studentScores = new int[10];
int total = 0;
double average;
```

4. Fill the studentScores array with values entered by the user, and accumulate the total in the same loop:

```
for (int i = 0; i < studentScores.length; i++) {
    System.out.print("Enter the score for student #" + (i + 1) + ": ");
    studentScores[i] = input.nextInt();
    total += studentScores[i];
}
```

5. Calculate the average by dividing the total by the number of elements in the studentScores array:

```
average = (double) total / studentScores.length;
System.out.println("Total score: " + total);
System.out.println("Average: " + average);
```

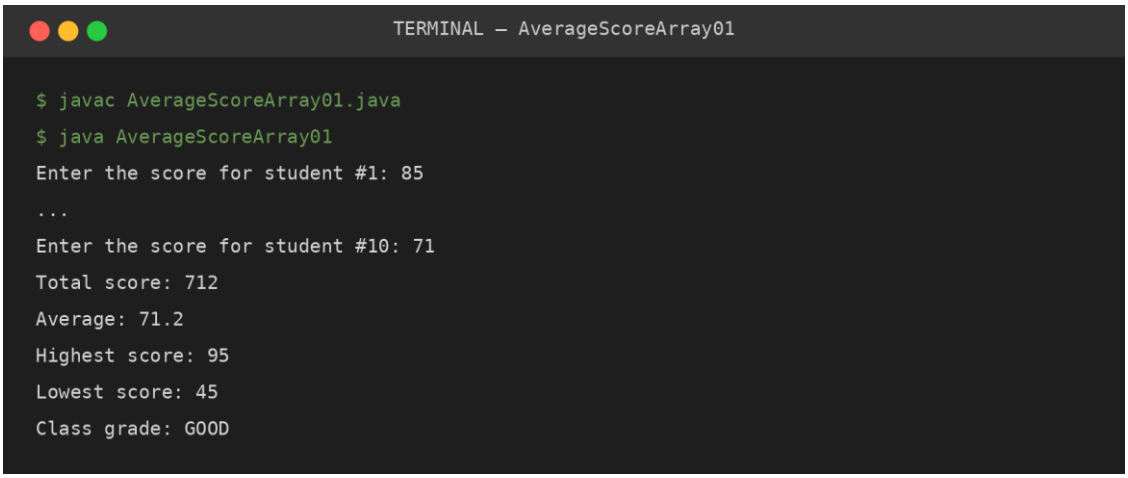
6. As an enrichment exercise, also find the highest and lowest values in the studentScores array using a for loop and comparison variables:

```
int highest = studentScores[0];
int lowest = studentScores[0];
for (int i = 1; i < studentScores.length; i++) {
    if (studentScores[i] > highest) highest = studentScores[i];
    if (studentScores[i] < lowest) lowest = studentScores[i];
}
System.out.println("Highest score: " + highest);
System.out.println("Lowest score: " + lowest);
```

7. Add a class grade category based on the average score, for example: >= 85 "EXCELLENT", >= 70 "GOOD", otherwise "NEEDS IMPROVEMENT", and then display that category.
8. Open the VS Code terminal, then compile and run the program with the following commands: javac **AverageScoreArray<RollNumber>.java** and then java **AverageScoreArray<RollNumber>**. Observe the results with several different sets of data.

Experiment 3 Sample Output (Reference)

The following is a sample output when the program is run with the same score data as in Experiment 2 (85, 60, 72, 90, 45, 78, 66, 95, 50, 71).



Example of the VS Code terminal display after the program is run with the sample data above.

Experiment 3 Questions

1. Modify the code above so that it displays the number of students who passed, that is, students whose score is greater than 70 (> 70).
2. Modify the program above so that it accepts the number of array elements from user input (not always 10), while still calculating the total, average, highest value, and lowest value correctly.
3. Why is (double) total / studentScores.length used in the average calculation instead of simply total / studentScores.length? Explain!

3.4. Experiment 4: Simple Searching and Sorting

Experiment Duration: 40 minutes

This experiment applies simple searching and sorting algorithms to student score data — two fundamental skills that are widely used in real-world data processing.

Part A — Linear Search

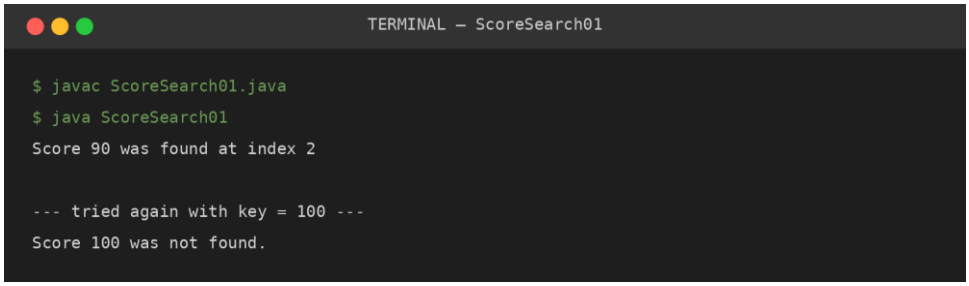
1. Open VS Code. Create a new file and save it with the name **ScoreSearch<RollNumber>.java**
2. Create an array named scores that contains student score data, then specify a value (*key*) to search for:

```
int[] scores = {78, 65, 90, 55, 88, 72};
int key = 90;
boolean found = false;

for (int i = 0; i < scores.length; i++) {
    if (scores[i] == key) {
        System.out.println("Score " + key + " was found at index " + i);
        found = true;
        break;
    }
}
if (!found) {
    System.out.println("Score " + key + " was not found.");
}
```

3. Open the VS Code terminal and run the program with the following commands: javac **ScoreSearch<RollNumber>.java** and then java **ScoreSearch<RollNumber>**. Observe the result, then try changing the value of key to one that does not exist in the array.

Part A Sample Output (Reference)



The first line shows the search result with key = 90 (found). The last line shows an example of the result when key is changed to a value that does not exist in the array, for example 100.

Part B — Bubble Sort

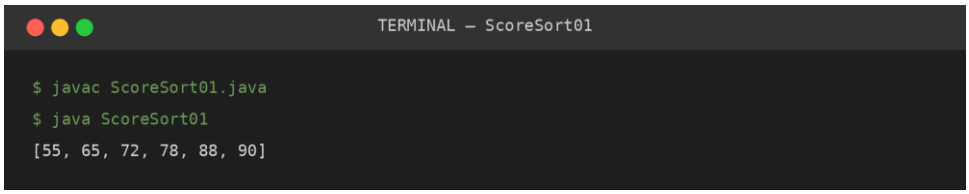
1. In VS Code, create a new file and save it with the name **ScoreSort<RollNumber>.java**
2. Use the same scores array as in Part A, then sort it from the smallest to the largest value using the bubble sort algorithm:

```
import java.util.Arrays;

int[] scores = {78, 65, 90, 55, 88, 72};
for (int i = 0; i < scores.length - 1; i++) {
    for (int j = 0; j < scores.length - 1 - i; j++) {
        if (scores[j] > scores[j + 1]) {
            int temp = scores[j];
            scores[j] = scores[j + 1];
            scores[j + 1] = temp;
        }
    }
}
System.out.println(Arrays.toString(scores));
```

3. Open the VS Code terminal, then compile and run the program with the following commands: javac **ScoreSort<RollNumber>.java** and then java **ScoreSort<RollNumber>**. Observe how the order of the array changes before and after the sorting process.

Part B Sample Output (Reference)



The array {78, 65, 90, 55, 88, 72} after being sorted from smallest to largest using bubble sort.

Experiment 4 Questions

1. Explain the meaning of the break; statement in the Part A code above!
2. Modify the Part A code so that the program accepts, as user input, the number of elements in the scores array, the array contents, and a value (*key*) to search for, and then prints the index position of that value (*key*) in the array.
3. Modify the Part A program so that it displays the message "The value you are looking for was not found" in a more informative format when the value being searched for (*key*) does not exist in the array.
4. Explain step by step how the *bubble sort* algorithm in Part B works, using the test data provided!
5. Modify the Part B program so that it sorts the array from the largest to the smallest value (*descending*), instead of from the smallest to the largest.

4. Assignments

Time Allocation: 80 minutes (to be completed at the end of the lab session, and may be continued outside lab hours if not yet finished)

4.1. Assignment 1 — Managing Student Scores

Write a program that can store and manage student scores (integers). The program must provide features to:

- Enter the number of student scores to be input.
- Enter each student score one by one.
- Calculate the average of all of the scores entered.
- Display the highest and lowest scores.
- Display all of the scores entered, along with their pass/fail status (PASS / FAIL).

4.2. Assignment 2 — Campus Café Ordering Application

Write a program that can manage food and beverage orders at a campus café. The program provides a predefined menu list, allows users to select menu items through a search feature, records orders, and calculates the total cost.

Café menu list (initialize an array of String at declaration):

```
String[] menu = {"Fried Rice", "Fried Noodles", "Toast", "French Fries", "Pulled Tea", "Cappuccino", "Chocolate Ice"};
int[] prices = {15000, 13000, 10000, 12000, 8000, 18000, 20000};
```

The program you write must meet the following requirements:

- Menu search feature: the user enters the name of the food or beverage they want to order, and the program searches for that name in the menu array using the linear search algorithm. If it is found, display its price; if it is not found, inform the user that the menu item is not available.
- Ordering feature: the user enters the number of items to order, then for each item selects a menu name (which must be validated to ensure it exists in the menu list). Store the names of the ordered items in a one-dimensional array that is separate from the main menu array.
- Calculate and display the total cost of all of the orders entered by the user.
- Display the list of orders that have been made, along with the total cost, in a neat and easy-to-read format.

Submission: Submit the program code (.java) for Assignment 1 and Assignment 2, together with screenshots of the program output, to the lecturer or lab assistant in accordance with the rules that apply in your respective class.